



UNIVERSIDAD TECNOLÓGICA NACIONAL

Materia: Programación 2

Proyecto: Trabajo Práctico Integrador – "Food Store"

Estudiante: Walter Exequiel Martínez

Fecha de Entrega: 20/06/26

Contents

UNIVERSIDAD TECNOLÓGICA NACIONAL.....	1
Marco Teórico.....	3
Decisiones Técnicas y Arquitectura del Sistema	4
Dificultades y Soluciones	8
Enlaces y Bibliografía.....	9

Marco Teórico

Este desarrollo de software se fundamenta en los pilares de la Programación Orientada a Objetos (POO) y las capacidades nativas del lenguaje Java. La POO nos permite modelar entidades del mundo real mediante abstracciones estructuradas, garantizando el cumplimiento de los siguientes principios:

- **Encapsulamiento:** Se aplica para restringir el acceso directo a los atributos de las entidades a través de modificadores de acceso privados (`private`), exponiendo su lectura y modificación controlada exclusivamente a través de métodos públicos de acceso con `getters` y `setters`.
- **Herencia y Abstracción:** El diseño usa una clase abstracta base denominada `Base`. Esta abstracción lo que hace es unificar los atributos de auditoría y control de estado de la aplicación (`id`, `eliminado`, `createdAt`), y nos va a permitir que las clases `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido` hereden de manera directa su comportamiento y estructura.
- **Polimorfismo:** Aparece mediante la implementación de la interfaz `Calculable`. Esta interfaz define un contrato de comportamiento único a través del método abstracto `calcularTotal()`, de esta forma, la clase `Pedido` tiene la responsabilidad de resolver la mecánica de cálculo matemático específico sobre su colección interna de detalles.

Ya que no teníamos un motor de bases de datos relacionales persistente, se utilizaron Colecciones como mecanismo de simulación de almacenamiento en memoria. Específicamente, se utiliza la estructura de datos `ArrayList`, la cual nos da un acceso posicional indexado rápido y dinámico para la manipulación CRUD de los registros durante el ciclo de vida de la ejecución del hilo principal de la aplicación.

Decisiones Técnicas y Arquitectura del Sistema

El sistema se estructuró bajo un patrón de Arquitectura en Capas, aislando por completo las responsabilidades y logrando una alta cohesión con un bajo acoplamiento

1. Capa de Modelo o Dominio (entities)

Contiene todas las clases puras del negocio. Los objetos se vinculan mediante relaciones de asociación reales. Por ejemplo, la entidad Producto tiene una referencia física directa a la entidad Categoría (relación N...1), evitando IDs planos y garantizando el tipado seguro. De la misma forma, la clase Pedido implementa una relación estricta de **Composición** hacia DetallePedido, lo que significa que los detalles están contenidos a la existencia de la cabecera del pedido.

2. Capa de Lógica de Negocio (services)

Centraliza las reglas comerciales del sistema y el control del estado. Los servicios (ProductoService, UsuarioService, etc.) coordinan las operaciones CRUD sobre las colecciones List. Aca es donde se ejecuta el principio de Soft Delete (baja lógica): los registros nunca se remueven físicamente de las colecciones usando remove(), en su lugar, se cambia el booleano eliminado a true. Esto nos asegura que los listados comerciales filtren los elementos inactivos a traves de el operador de negación (!objeto.isEliminado()), pero también se resguarda la integridad de los datos históricos en el caso de auditorías o consultas de pedidos previos.

3. Capa de Interfaz de Usuario (Main.java)

Usa la clase Scanner para la captura de flujos de texto de la consola y se encarga del renderizado estético de los menús e interacción con el usuario. El Main no tiene reglas operativas complejas, simplemente consume los servicios de la capa lógica y gestiona el enrutamiento de flujos de las operaciones numéricas.

La robustez de la aplicación contra fallos de operación o entradas corruptas del usuario se reforzó a través de dos estrategias principales:

1. Métodos de Lectura de Consola Seguros

Para suavizar los fallos por ingreso de datos alfabéticos en variables numéricas (lo que provocaría el colapso instantáneo del programa por excepciones genéricas del sistema), se desarrollaron funciones de tipado seguro en la interfaz: leerEntero, leerDouble y leerLong. Estos métodos lo que hacen es encapsular un bucle de control infinito que atrapa la entrada y aplica bloques try-catch específicos sobre la excepción de formato de número (NumberFormatException). Si el usuario introduce una cadena inválida, el error se contiene y se solicita el dato nuevamente de forma limpia.

2. Implementación de Excepciones Propias

Se diseñaron excepciones personalizadas controladas que heredan de la clase base Exception. El control de transacciones críticas del negocio se divide en:

- EmailDuplicadoException: Lanzada por UsuarioService por si se llega a detectar que un correo electrónico ya existe registrado en una cuenta activa de la colección en memoria.
- StockInsuficienteException: PedidoService la usa de forma preventiva por si la cantidad solicitada de un ítem supera el inventario físico disponible en ProductoService, así anula la creación completa del carrito para evitar saldos inconsistentes.
- CategoriaEnUsoException: Esta es para proteger la Integridad Referencial del sistema, bloquea la baja lógica de una categoría si existen productos activos “mapeados” a esa categoría.

Intentando crear una Categoría sin nombre, lo cual nos devuelve un Error, aviándonos que el nombre no puede estar vacío

```
=== CATEGORIAS ===
1. Listar categorias
2. Crear nueva categoria
3. Editar categoria
4. Eliminar categoria
0. Volver al menu principal
Seleccione: 2

=== NUEVA CATEGORIA ===
Ingresa el nombre de la categoria:
Error: El nombre no puede estar vacío
```

Aca los métodos leerDouble y leerEntero atrapan el error a través de try-catch al intentar ingresar el precio (o el stock) como cadena, ya que obliga a colocar un decimal o entero para el caso del Stock

```
=== NUEVO PRODUCTO ===
Ingresa el nombre del producto: Hamburguesa
Ingresa la descripcion: Hamburguesa Vegana
Ingresa el precio: cinco mil
Error: Tienes que ingresar un número decimal válido (Ejemplo: 550.50, intenta nuevamente.
Ingresa el precio:
```

```
=== PRODUCTOS ===  
ID: 1  
Producto: Hamburguesa  
Precio: $5000.0  
Stock: 3  
Imagen:  
Disponibilidad: Disponible  
Categoria: Hamburguesas  
Estado: Activo
```

```
=== NUEVO PEDIDO ===  
Ingresa el ID del usuario(Cliente): 1  
Ingresa el ID del producto: 1  
Producto seleccionado: Hamburguesa  
Precio: $5000.0  
Ingresa la cantidad: 5  
Producto añadido al carrito  
Desea agregar otro producto? (S/N)  
N  
Selecciona la Forma de Pago (1: EFECTIVO, 2: TARJETA, 3: TRANSFERENCIA): 1  
Error en la transaccion: Stock insuficiente para el producto: Hamburguesa
```

Aca podemos ver que “Hamburguesa” tiene un Stock de 3 unidades, por lo que al realizar un pedido e intentar comprar mas de la cantidad disponible, da un error de Stock insuficiente

Dificultades y Soluciones

Durante las etapas de integración de componentes, me surgieron tres obstáculos técnicos clave que requirieron refactorizaciones:

1. **Menús Anidados:** Al coordinar operaciones mas complejas de transacciones de pedidos, detecte un desvío de flujo donde el programa avanzaba de forma automática e involuntaria hacia el caso de edición al finalizar una acción correcta. Esto lo pude solucionar colocando de forma obligatoria las sentencias de corte break; al final de cada bloque del switch.
2. **Validaciones Tardías:** Al principio, el sistema capturaba de corrido todas las variables de un registro y evaluaba las cadenas vacías en un bloque if-else al final de la operación. Esto obligaba a completar datos para luego rechazar la inserción. Lo pude resolver cambiando el flujo hacia un paradigma de validación temprana: apenas se procesa el campo de identidad principal, se evalúa su longitud con `.isEmpty()` y, ante una falla, se rompe el flujo del menú al instante.
3. **Filtrados Invertidos en Listados:** Con los primeros testeos del inventario de productos, la consola ocultaba los artículos activos y mostraba únicamente los elementos dados de baja. Despues de revisar la lógica del método `listar()` en `ProductoService`, me di cuenta de una inversión lógica en el condicional evaluador. Lo corregí anteponiendo el operador de negación lógica (!) sobre el método consultor del estado de la entidad.

Enlaces y Bibliografía

Video Demostrativo: <https://youtu.be/WV6nPd0Pjwg>

Repositorio del Proyecto: <https://github.com/M-exe97/UTN-TUPaDProgramacion2-Trabajo-Integrador>

[@CharlyCimino][YouTube] <https://www.youtube.com/@CharlyCimino>

Programación Orientada a Objetos 2: Profundización de temas [PDF]. Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional.

Programación II: Colecciones [PDF]. Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional.

Programación II: Herencia y Polimorfismo [PDF]. Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional.

Programación II: Interfaces y Manejo de Errores [PDF]. Tecnicatura Universitaria en Programación, Universidad Tecnológica Nacional.